HW5. Heat equations

60212440 박기성

Problem

Source term을 포함하는 2차원 Heat equation이 다음과 같이 주어진다.

$$\frac{\partial \phi}{\partial t} = \alpha\left(\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2}\right) + S(x,y)\,, \qquad -1 \le x \le 1, -1 \le y \le 1.$$

균일한 초기 및 경계조건은 $\phi(x,y,0) = 0, \phi(\pm 1, y, t) = 0, \phi(x, \pm 1, t) = 0$ 이며, 본 문제에서 열전도율 α는 1로 주어진다.

Problem (1) Source term $S(x,y) = 2(2 - x^2 - y^2)$일 때, ϕ의 대한 exact solution을 구하시오.

- 정상상태에서 $\frac{\partial \phi}{\partial t} = 0$이고, $\alpha = 1$이므로

$$\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} + S(x,y) = 0$$

$$\nabla^2 \phi = -S(x,y) = -2(2 - x^2 - y^2)$$

$$\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = -2(2 - x^2 - y^2)$$

Problem (1)

- 균일 초기 및 경계조건

$\phi(x,y,0)=0$, $\phi(\pm 1,y,t)=0, \phi(x,\pm 1,y,t)=0$ 을 동시 만족하도록

$$\phi(x,y)=A(1-x^2)(1-y^2)\text{으로 가정.}$$

$$\frac{\partial^2\phi}{\partial x^2}=A\frac{\partial^2}{\partial x^2}(1-x^2)(1-y^2)=A(-2)(1-y^2)$$

$$\frac{\partial^2\phi}{\partial y^2}=A\frac{\partial^2}{\partial y^2}(1-y^2)(1-x^2)=A(-2)(1-x^2)$$

Problem (1)

$$\nabla^2\phi = -\mathrm{S}(\mathrm{x},\mathrm{y}) = -2A + 2Ay^2 - 2A + 2Ax^2 = -2(2 - x^2 - y^2)$$
$$= -4A + 2Ax^2 + 2Ay^2 = -4A + 2x^2 + 2y^2$$

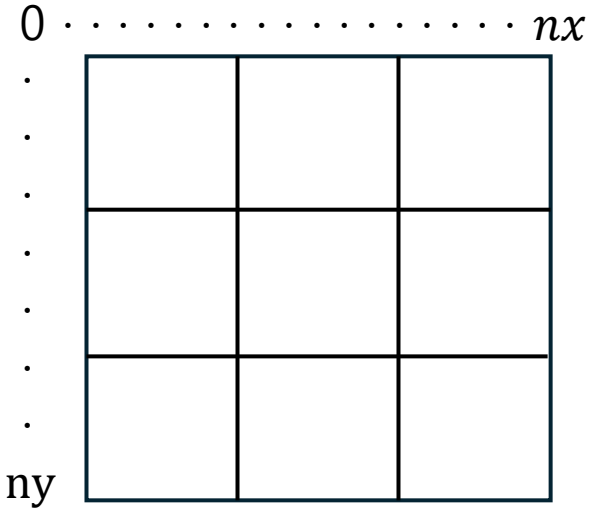
$$A = 1$$

$$\therefore \phi(x,y) = (1 - x^2)(1 - y^2) = x^2y^2 - x^2 - y^2 + 1$$

Problem (2)

2. uniform 격자계에서 시간에 대하여 Crank-Nicolson method를, 공간에 대하여 2차 central difference scheme을 사용하여 정상상태 (steady state)에 도달하도록 방정식을 푸시오. Exact solution과 수치해석한 정상상태의 solution을 시간간격 Δt와 x, y 방향 격자수 (각각 N, M)에 변화를 주어 plot하시오.

$$\frac{\phi_{i,j}^{n+1} - \phi_{i,j}^{n}}{\Delta t} = \frac{1}{2}(\mathrm{f^{n+1}} + \mathrm{f^{n}})$$



총 노드 수
(Nx+1)x(Ny+1)

Problem (2)

$$\frac{\phi_{i,j}^{n+1}-\phi_{i,j}^{n}}{\Delta t}=\frac{1}{2}\alpha\left(\nabla^2\phi_{i,j}^{n}+\nabla^2\phi_{i,j}^{n+1}\right)+\frac{1}{2}\left(S_{i,j}^{n+1}+S_{i,j}^{n}\right)$$

$$\frac{\partial^2\phi}{\partial x^2}\approx\frac{\phi_{i+1,j}-2\phi_{i,j}+\phi_{i-1,j}}{(\Delta x)^2}$$

$$\frac{\partial^2\phi}{\partial y^2}\approx\frac{\phi_{i,j+1}-2\phi_{i,j}+\phi_{i,j-1}}{(\Delta y)^2}$$

Problem (2)

$$\frac{\phi_{i,j}^{n+1}-\phi_{i,j}^{n}}{\Delta t}=\frac{1}{2}\alpha\left[\left(\frac{\phi_{i+1,j}^{n+1}-2\phi^{n+1}{}_{i,j}+\phi_{i-1,j}^{n+1}}{(\Delta x)^2}+\frac{\phi_{i,j+1}^{n+1}-2\phi_{i,j}^{n+1}+\phi_{i,j-1}^{n+1}}{(\Delta y)^2}\right)+\left(\frac{\phi_{i+1,j}^{n}-2\phi^{n}{}_{i,j}+\phi_{i-1,j}^{n}}{(\Delta x)^2}+\frac{\phi_{i,j+1}^{n}-2\phi_{i,j}^{n}+\phi_{i,j-1}^{n}}{(\Delta y)^2}\right)\right]+\frac{1}{2}(S_{i,j}^{n+1}+S_{i,j}^{n})$$

Uniform 격자계 $\Delta x=\Delta y=h \rightarrow \beta=\frac{\alpha\Delta t}{2h^2}$

$$\frac{\phi_{i,j}^{n+1}-\phi_{i,j}^{n}}{\Delta t}=\frac{\alpha}{2h^2}\left[(\phi_{i+1,j}^{n+1}+\phi_{i-1,j}^{n+1}+\phi_{i,j+1}^{n+1}+\phi_{i,j-1}^{n+1}-4\phi^{n+1}{}_{i,j})+(\phi_{i+1,j}^{n}+\phi_{i-1,j}^{n}+\phi_{i,j+1}^{n}+\phi_{i,j-1}^{n}-4\phi^{n}{}_{i,j})\right]+\frac{1}{2}(S_{i,j}^{n+1}+S_{i,j}^{n})$$

$$\phi_{i,j}^{n+1}-\beta((\phi_{i+1,j}^{n+1}+\phi_{i-1,j}^{n+1}+\phi_{i,j+1}^{n+1}+\phi_{i,j-1}^{n+1}-4\phi_{i,i}^{n+1})=\phi_{i,j}^{n}+\beta\left(\phi_{i+1,j}^{n}+\phi_{i-1,j}^{n}+\phi_{i,j+1}^{n}+\phi_{i,j-1}^{n}-4\phi^{n}{}_{i,j}\right)+\frac{1}{2}\left(S_{i,j}^{n+1}+S_{i,j}^{n}\right)\cdot\Delta t$$

Problem (2)

$$\left(I-\beta L_x-\beta \mathrm{L_y}\right)\phi_{\mathrm{i,j}}^{\mathrm{n+1}}=\left(\mathrm{I}+\beta \mathrm{L_x}+\beta \mathrm{L_y}\right)\phi_{\mathrm{i,j}}^{\mathrm{n}}+\frac{1}{2}\left(\mathrm{S^{n+1}}+\mathrm{S^{n}}\right)\cdot \Delta \mathrm{t}$$

$$\left(I-\beta \mathrm{L_x}\right)\left(I-\beta \mathrm{L_y}\right)\phi_{\mathrm{i,j}}^{\mathrm{n+1}}-\beta^2 \mathrm{L_x L_y}\phi_{\mathrm{i,j}}^{\mathrm{n+1}}$$
$$=\left(I+\beta \mathrm{L_x}\right)\left(\mathrm{I}+\beta \mathrm{L_y}\right)\phi_{\mathrm{i,j}}^{\mathrm{n}}+\frac{\Delta \mathrm{t}}{2}\left(\mathrm{S^{n+1}}+\mathrm{S^{n}}\right)-\beta^2 \mathrm{L_x L_y}\phi_{\mathrm{i,j}}^{\mathrm{n}}$$

$$\underbrace{\left(I-\beta \mathrm{L_x}\right)\left(\mathrm{I}-\beta \mathrm{L_y}\right)\phi_{\mathrm{i,j}}^{\mathrm{n+1}}}_{\Psi}=\underbrace{\left(I+\beta \mathrm{L_x}\right)\left(\mathrm{I}+\beta \mathrm{L_y}\right)\phi_{\mathrm{i,j}}^{\mathrm{n}}+\frac{\Delta \mathrm{t}}{2}\left(\mathrm{S^{n+1}}+\mathrm{S^{n}}\right)}_{\mathrm{R}}$$

Problem (2)

$$-\beta^2 \mathrm{L_x L_y}(\phi_{\mathrm{i,j}}^{\mathrm{n+1}} - \phi_{i,\mathrm{j}}^{\mathrm{n}}) \sim > error \sim O(\Delta t^2)\ (\because \beta = \frac{\alpha \Delta t}{2\Delta \mathrm{x}^2})$$

이항은 실제로 $O(\Delta t^3)$이지만, 전체 $scheme$이 2차 정확도이므로 $O(\Delta t^2)$이하의 절단오차로 취급함.

(Crank-Nicolson법과 2차 중앙차분법은 모두 2차 정확도를 가짐)

Problem (2)

$$\underbrace{\left(I-\beta \mathrm{L}_y\right)\left(\mathrm{I}-\beta \mathrm{L}_x\right)\phi_{\mathrm{i,j}}^{\mathrm{n+1}}}_{\Psi}=\underbrace{\left(I+\beta \mathrm{L}_{\mathrm{x}}\right)\left(\mathrm{I}+\beta \mathrm{L}_{\mathrm{y}}\right)\phi_{\mathrm{i,j}}^{\mathrm{n}}+\frac{\Delta \mathrm{t}}{2}\left(\mathrm{S}^{\mathrm{n+1}}+\mathrm{S}^{\mathrm{n}}\right)}_{\mathrm{R}}$$

$$A \cdot X = b$$

(1) $\left(I-\beta L_y\right)\psi_{i,j}=R_{i,j}$: y방향에 대해 implicit하게 푸는 과정

(2) $\left(I-\beta L_x\right)\phi_{i.j}^{n+1}=\psi_{i,j}$: x방향에 대해 implicit하게 푸는 과정

전체 선형 시스템을 두 번에 나누어 푸는 방식: ADI방식(Alternating Direction Implicit)

Problem (2)

$$L_x\phi_{i,j} = \phi_{i-1,j} - 2\phi_{i,j} + \phi_{i+1,j}\ (for\ fixed\ j)$$

$$L_y\phi_{i,j} = \phi_{i,j-1} - 2\phi_{i,j} + \phi_{i,j+1}\ (for\ fixed\ i)$$

$$\text{ex) } L_X = \begin{bmatrix} -2 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 1 & -2 \end{bmatrix}$$

Problem (2)

```python
import numpy as np
import matplotlib.pyplot as plt

def S(x,y):
    s = 2*(2-x**2-y**2)
    return s

#parameters
alpha = 1
N = M = 20 # 격자 수

# 격자 생성
x = np.linspace(-1,1,N)
y = np.linspace(-1,1,M)

# 격자 좌표 생성
X,Y = np.meshgrid(x,y)

h = x[1] - x[0] # 격자 간격

t = 0 #initial time
dt = 0.01 #time step

beta = alpha * dt / (2*h**2)
```

X,Y = meshgrid(x,y)

x,y는 1차원 배열
meshgrid()는 x축과 y축 좌표를 가지고, 격자판 위의 모든 점의 위치를 (x,y)형태로 자동을 만들어줌.

X[i,j] : x_j값-> x좌표만 저장한 2차원 배열
Y[i,j] : y_i값-> y좌표만 저장한 2차원 배열
(X[i,j] , Y[i,j]) = 격자점 (x_j, y_i)의 실제 좌표

$$\beta = \frac{\alpha \Delta t}{2h^2}$$

Problem (2)

```
# 초기 phiφ 설정
phi = np.zeros((N, N)) # 초기값 설정
I = np.eye(N-2) # 내부 격자에 대한 단위 행렬


# 내부 격자에 대한 행렬 (N-2)x(N-2)
L = np.zeros((N-2, N-2))
for i in range(N-2):
    L[i,i] = -2
    if i-1 >= 0:
        L[i,i-1] = 1
    if i+1 < N-2:
        L[i,i+1] = 1

# 행렬 A 정의
A = I - beta * L
```

Phi를 NxN 격자 전체를0으로 초기화.
- 경계조건 $x = \pm 1, y = \pm 1$ 에서 0, 즉 경계값 0

계산은 경계를 제외한 내부 격자점에서만 이루어짐.

단위 행렬을 경계를 제외한 (N-2)X(N-2) 크기로 생성.

$$\text{ex) } L_x = \begin{bmatrix} -2 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 1 & -2 \end{bmatrix}$$

Problem (2)

```
# 시간에 따른 phiφ값을 저장할 리스트
phi_list = [phi,]

# 시각화할 인덱스 선택 (1~149 중 10개)
j_list = np.linspace(1,149,10, dtype=int)

for j in range(150):
    t = t + dt

    R = S(X[1:-1,1:-1], Y[1:-1,1:-1])*dt + (I + beta * L) @ ((I + beta * L) @ phi_list[j][1:-1, 1:-1].T).T
    psi = np.linalg.solve(A, R)  # solve the linear system
    phi_new = np.dot(psi, np.linalg.inv(A.T))

    # 전체 phi 행렬에 내부 영역만 채워 넣음
    phi_full = np.zeros((N, N))  # initialize full pi
    phi_full[1:-1, 1:-1] = phi_new
    phi_list.append(phi_full)
```

$$S^{n+1} = S^n \rightarrow \frac{1}{2}(S^{n+1} + S^n) = \mathrm{S} \quad (\because S(x,y)\ )$$

Problem (2)

```
R = S(X[1:-1,1:-1], Y[1:-1,1:-1])*dt + (I + beta * L) @ ((I + beta * L) @ phi_list[j][1:-1, 1:-1].T).T
psi = np.linalg.solve(A, R)  # solve the linear system
phi_new = np.dot(psi, np.linalg.inv(A.T))
```

- phi_list[j][1:-1, 1:-1]로 내부 격자만 꺼내 전치(.T)
- (I+beta*L) @ phi_list [j][1:-1, 1:-1].T : y방향 implicit 계산
- (I+beta*L) @ (y방향 계산결과).T : x방향 implicit 계산
- @ 행렬곱

Problem (2)

```
for j in range(150):
    t = t + dt

    R = S(X[1:-1,1:-1], Y[1:-1,1:-1])*dt + (I + beta * L) @ ((I + beta * L) @ phi_list[j][1:-1, 1:-1].T).T
    psi = np.linalg.solve(A, R)  # solve the linear system
    phi_new = np.dot(psi, np.linalg.inv(A.T))

    # 전체 phi 행렬에 내부 영역만 채워 넣음
    phi_full = np.zeros((N, N))  # initialize full pi
    phi_full[1:-1, 1:-1] = phi_new
    phi_list.append(phi_full)
```

Problem (2)

```python
# phi 전체 범위 계산 (루프 이후)
phi_min = np.min([np.min(p) for p in phi_list])
phi_max = np.max([np.max(p) for p in phi_list])

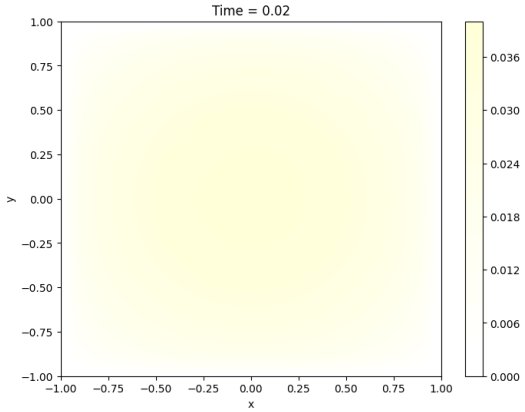
# 시각화
for j in j_list:
    plt.figure(figsize=(8,6))
    plt.contourf(X, Y, phi_list[j], levels=20, cmap='hot_r', vmin=phi_min, vmax=phi_max)
    plt.colorbar()
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title(f'Time = {j*dt:.2f}')
    plt.show()

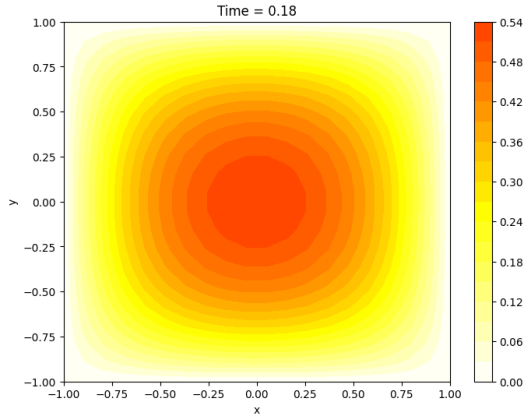
def phi_exact(x,y):
    return (1-x**2)*(1-y**2)

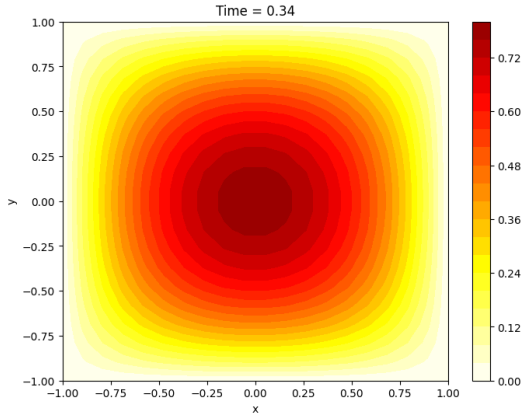
# exact solution 시각화
plt.figure(figsize=(8, 6))
plt.contourf(X, Y, phi_exact(X,Y), levels=20, cmap='hot_r', vmin=0, vmax=1)
plt.colorbar(label='φ exact')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Exact Solution')
plt.show()
```

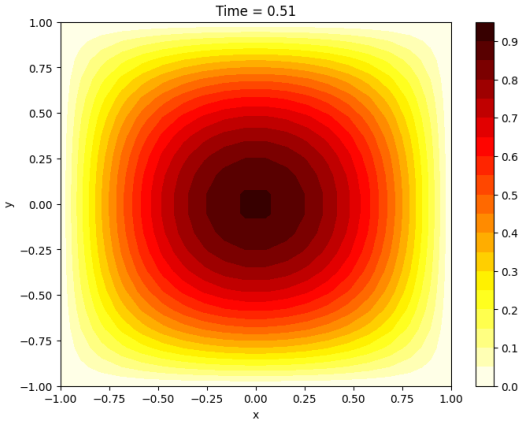
Problem (2)

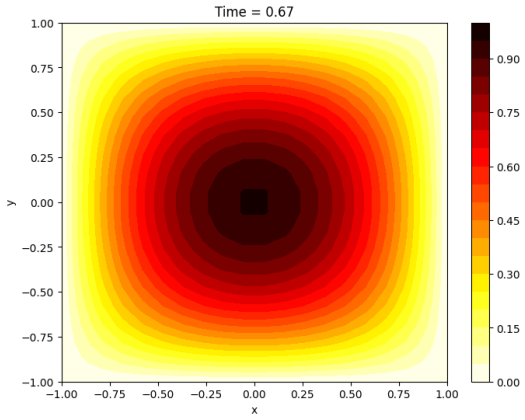
N = 20 dt= 0.01

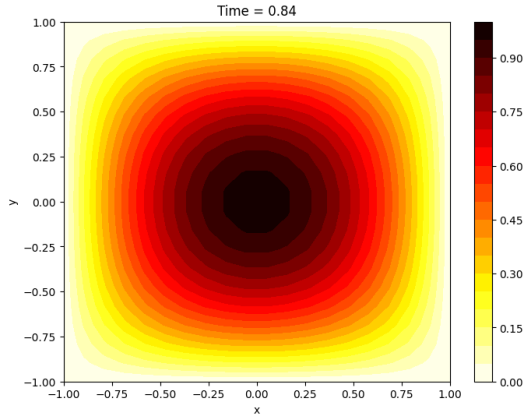


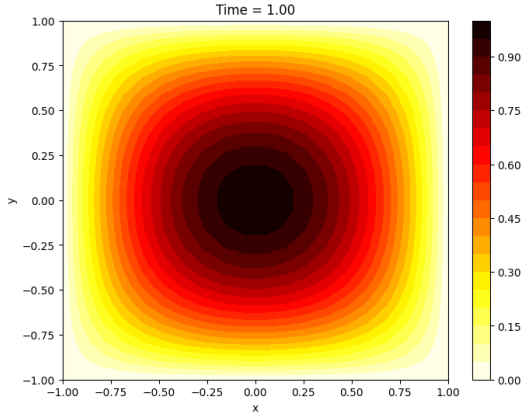


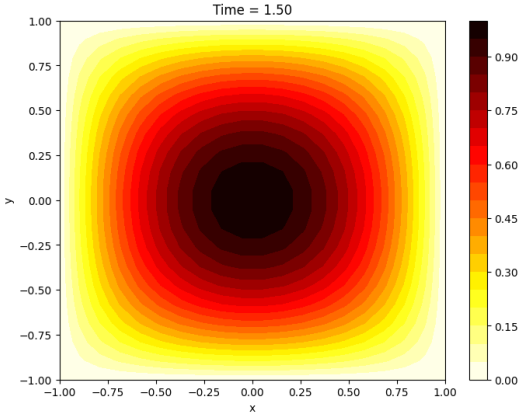




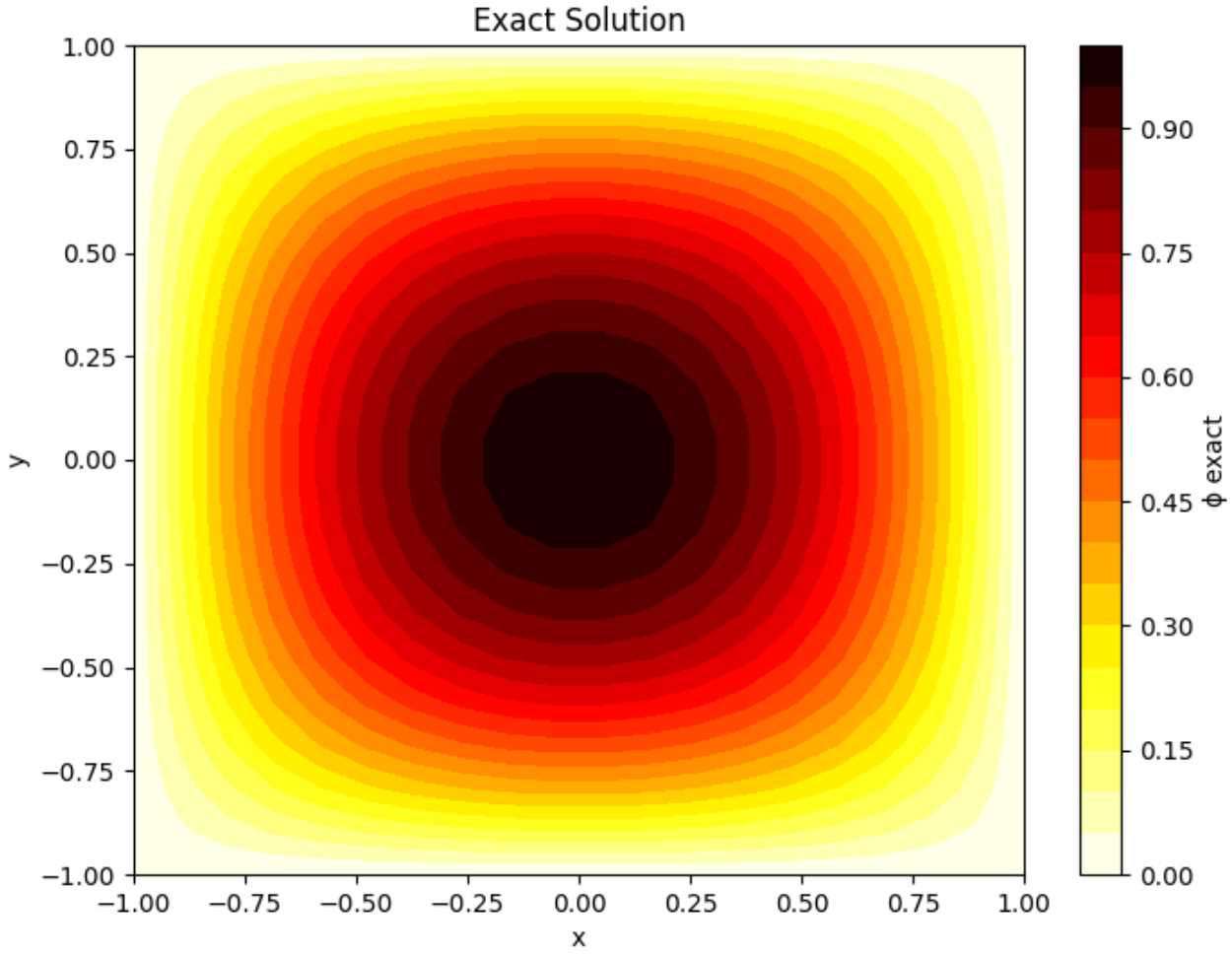


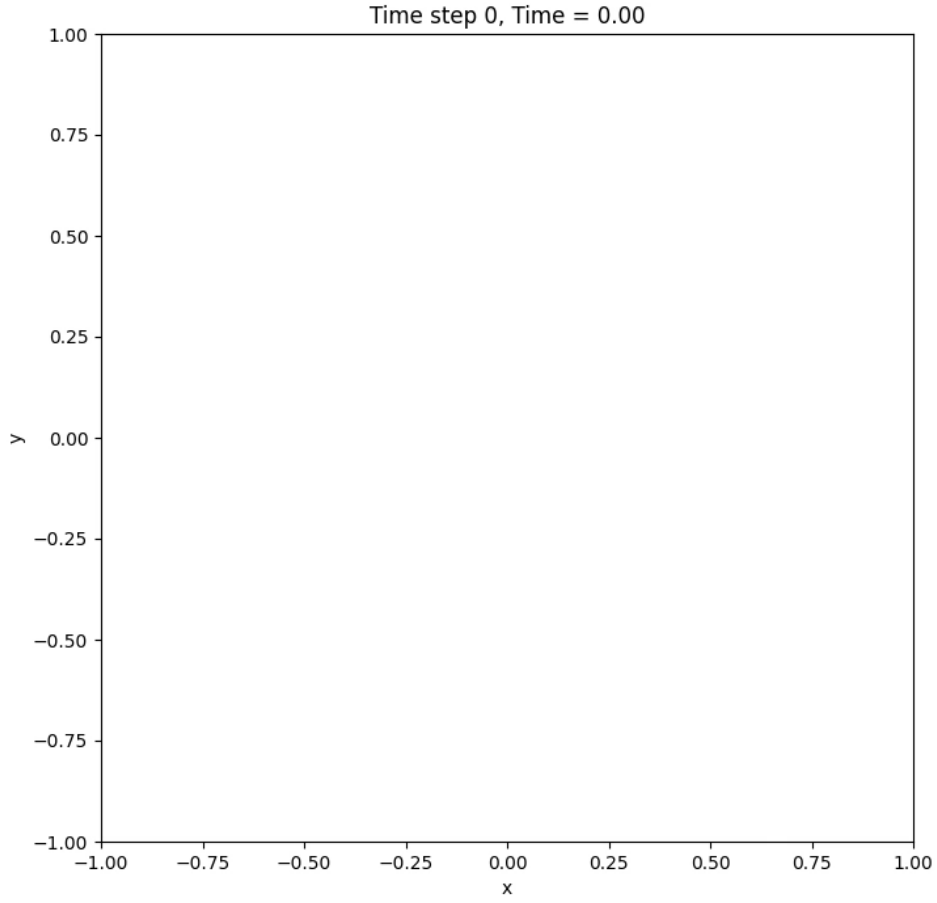






Problem (2)


Exact Solution
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
x
0.90
0.75
0.60
0.45
0.30
0.15
0.00
ϕ exact


Time step 0, Time = 0.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
x

Problem (3)

- 수치해석 결과의 order of accuracy를 시간과 공간에 대하여 분석하시오.

Problem (3) Space

```
# Space
h_list = []
error_list = []

for N in range(5,20,1):

    alpha = 1
    x_list = np.linspace(-1, 1, N) # x grid points
    y_list = np.linspace(-1, 1, N) # y grid points
    X, Y = np.meshgrid(x_list, y_list) # create a meshgrid
    h = x_list[1] - x_list[0] # grid spacing
    dt = 0.005 # time step
    t = 0 # initial time
    beta = alpha * dt / (h**2) / 2

    phi = np.zeros((N, N)) # initialize pi
    I = np.eye(N-2) # identity matrix
    L = np.zeros((N-2, N-2)) # initialize pi

    phi_list = [phi,]
```

- dt =0.005

Problem (3) Space

```
for i in range(N-2):
    L[i, i] = -2  # 주대각선

    if i - 1 >= 0:
        L[i, i-1] = 1  # 왼쪽 이웃

    if i + 1 < (N-2):
        L[i, i+1] = 1  # 오른쪽 이웃

A = (I - beta * L)

for j in range(1000):
    t = t + dt

    R = S(X[1:-1,1:-1], Y[1:-1,1:-1])*dt \
        + (I + beta * L) @ ((I + beta * L) @ phi_list[j][1:-1, 1:-1].T).T
    psi = np.linalg.solve(A, R)  # solve the linear system
    phi_new = np.dot(psi, np.linalg.inv(A.T))
    phi_full = np.zeros((N, N))  # initialize full pi
    phi_full[1:-1, 1:-1] = phi_new
    phi_list.append(phi_full)

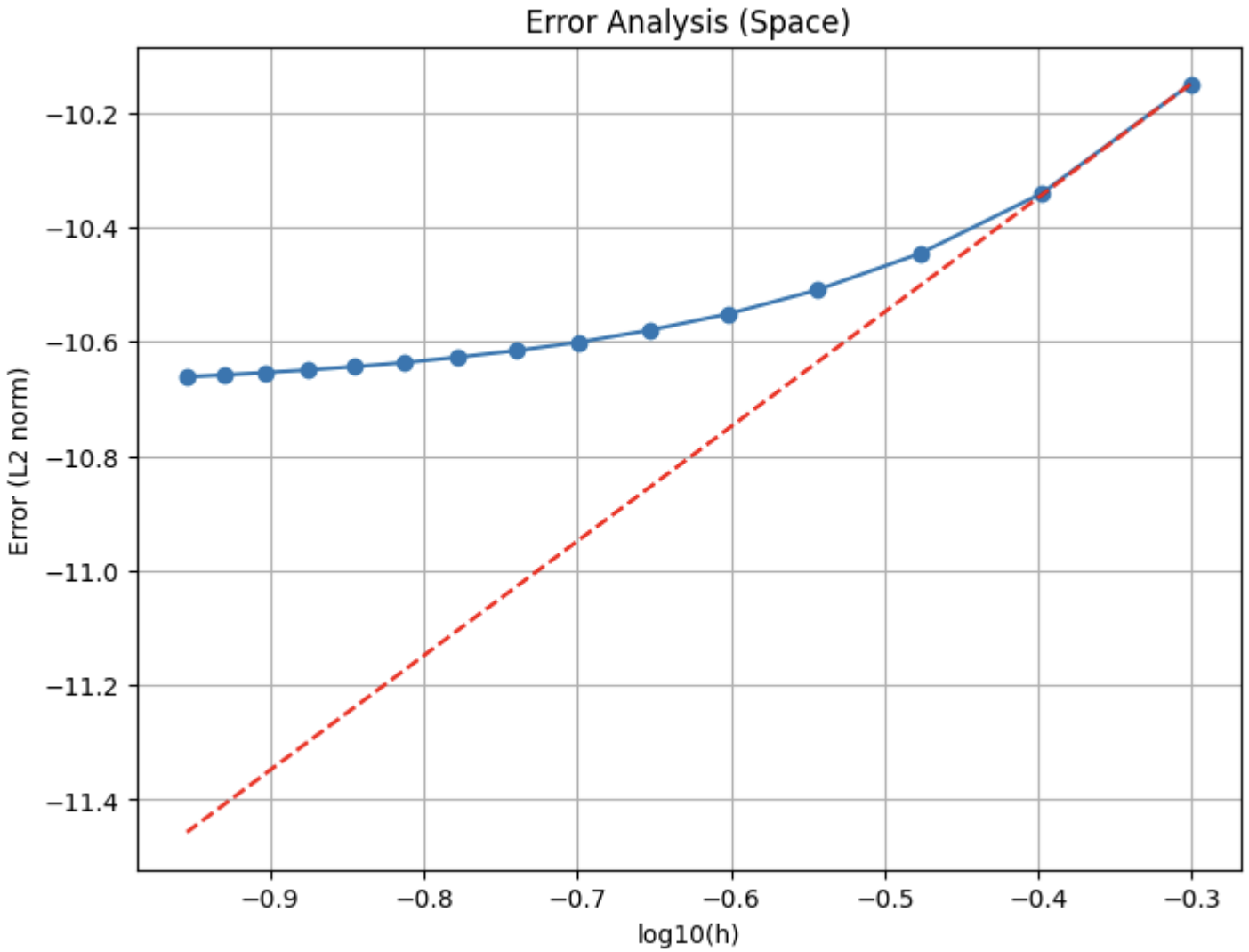
error_list.append(np.linalg.norm(phi_list[-1] - phi_exact(X, Y), 2)*h)
h_list.append(h)
```

- $\|e\|_{L^2} = \left(\int_\Omega |\phi_{num}(x,y) - \phi_{exact}|^2 dxdy\right)^{\frac{1}{2}}$
- $\|e\|_{L^2} \approx \left(\sum_{i,j} |e_{i,j}|^2 h^2\right)^{\frac{1}{2}}$
- Linalg.norm(~,2) $= \sqrt{\sum e_{i,j}^2}$

Problem (3) Space

```
plt.figure(figsize=(8,6))
plt.plot(np.log10(h_list), np.log10(error_list), marker='o', label='Error')
plt.xlabel('log10(h)')
plt.ylabel('Error (L2 norm)')
plt.plot(np.log10(h_list), 2*(np.log10(h_list)-np.log10(h_list[0]))+np.log10(error_list[0]), linestyle='--', color='r', label='Slope 2')
plt.title('Error Analysis (Space)')
plt.grid()
```

Problem (3) Space



$$E(h, dt) = O(\Delta t^2 + h^2)$$

Problem (3) time

```python
#time
t_list = []
error_list = []
N = M = 20  # Fixed grid size for time analysis
t_final = 3

for p in range(5, 11, 1):
    alpha = 1
    x_list = np.linspace(-1, 1, N)  # x grid points
    y_list = np.linspace(-1, 1, M)  # y grid points
    X, Y = np.meshgrid(x_list, y_list)  # create a meshgrid
    h = x_list[1] - x_list[0]  # grid spacing
    dt = 0.006* ( p**2 ) # time step, increasing with p
    steps = int(t_final/dt) # dt에 따른 step 수 조정
    beta = alpha * dt / (h ** 2) / 2

    phi = np.zeros((N, M))  # initialize phi
    I = np.eye(N - 2)  # identity matrix
    L = np.zeros((N - 2, N - 2))  # initialize L

    phi_list = [phi, ]
```

- t_final = 3

```python
print(t_list)
print(np.log10(t_list))
print(steps)
```

✓ 0.0s

```
[0.15, 0.216, 0.294, 0.384, 0.486, 0.6]
[-0.82390874 -0.66554625 -0.53165267 -0.41566878 -0.31336373 -0.22184875]
5
```

Problem (3) time

```
for i in range(N - 2):
    L[i, i] = -2  # main diagonal

    if i - 1 >= 0:
        L[i, i - 1] = 1  # left neighbor

    if i + 1 < (N - 2):
        L[i, i + 1] = 1  # right neighbor

A = (I - beta * L)

for j in range(steps):
    t = t + dt

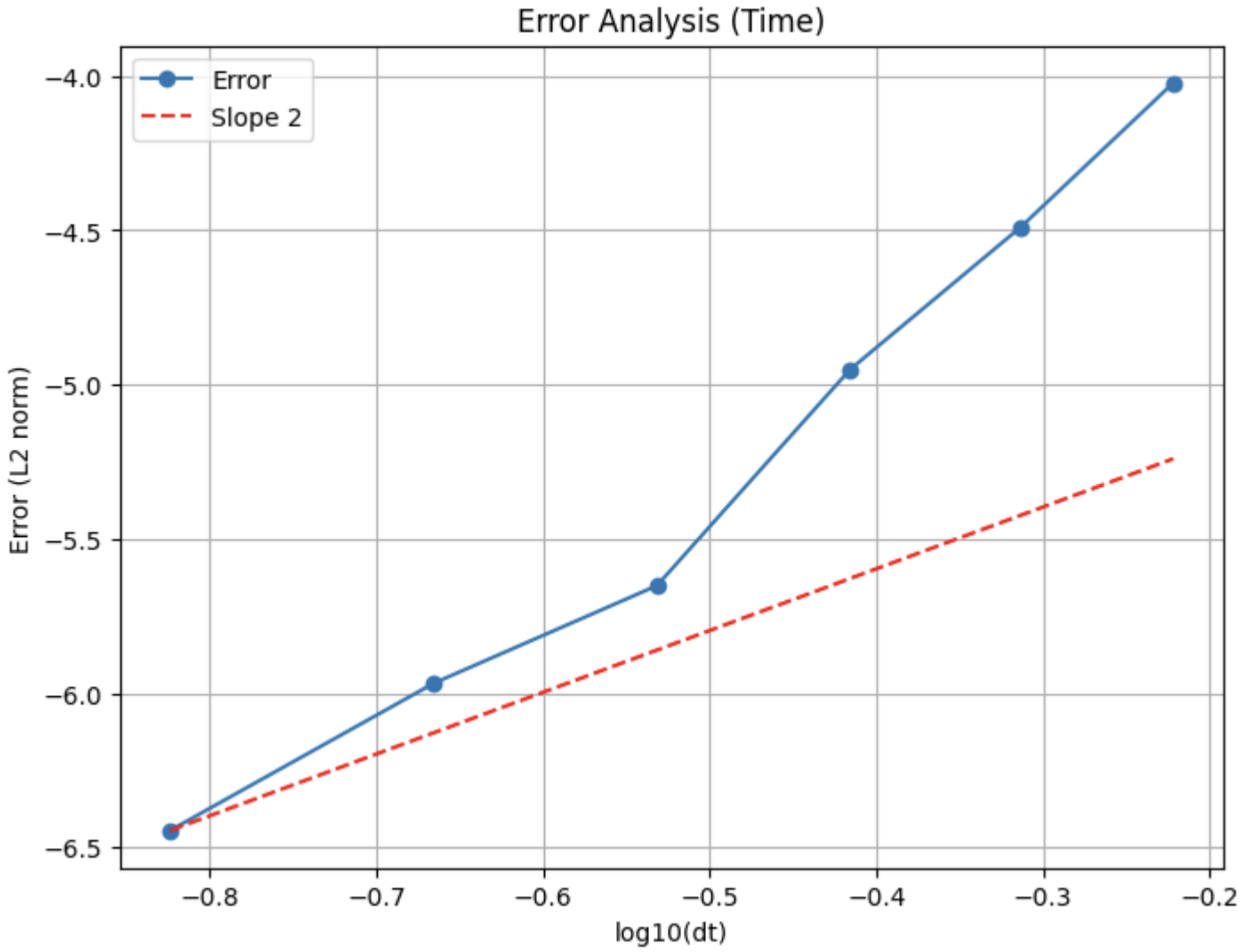
    R = S(X[1:-1,1:-1], Y[1:-1,1:-1])*dt \
    + (I + beta * L) @ ((I + beta * L) @ phi_list[j][1:-1, 1:-1].T).T
    psi = np.linalg.solve(A, R)  # solve the linear system
    phi_new = np.dot(psi, np.linalg.inv(A.T))
    phi_full = np.zeros((N, M))  # initialize full pi
    phi_full[1:-1, 1:-1] = phi_new
    phi_list.append(phi_full)

error_list.append(np.linalg.norm(phi_list[-1] - phi_exact(X, Y), 2) * h)
t_list.append(dt)  # store time step
```

Problem (3) time

```
# analysis order of accuracy
plt.figure(figsize=(8, 6))
plt.plot(np.log10(t_list), np.log10(error_list), marker='o', label='Error')
plt.xlabel('log10(dt)')
plt.ylabel('Error (L2 norm)')
plt.plot(np.log10(t_list), 2 * (np.log10(t_list) - np.log10(t_list[0])) + np.log10(error_list[0]), linestyle='--', color='r', label='Slope 2')
plt.title('Error Analysis (Time)')
plt.grid()
plt.legend()
plt.show()
```

Problem (3) time

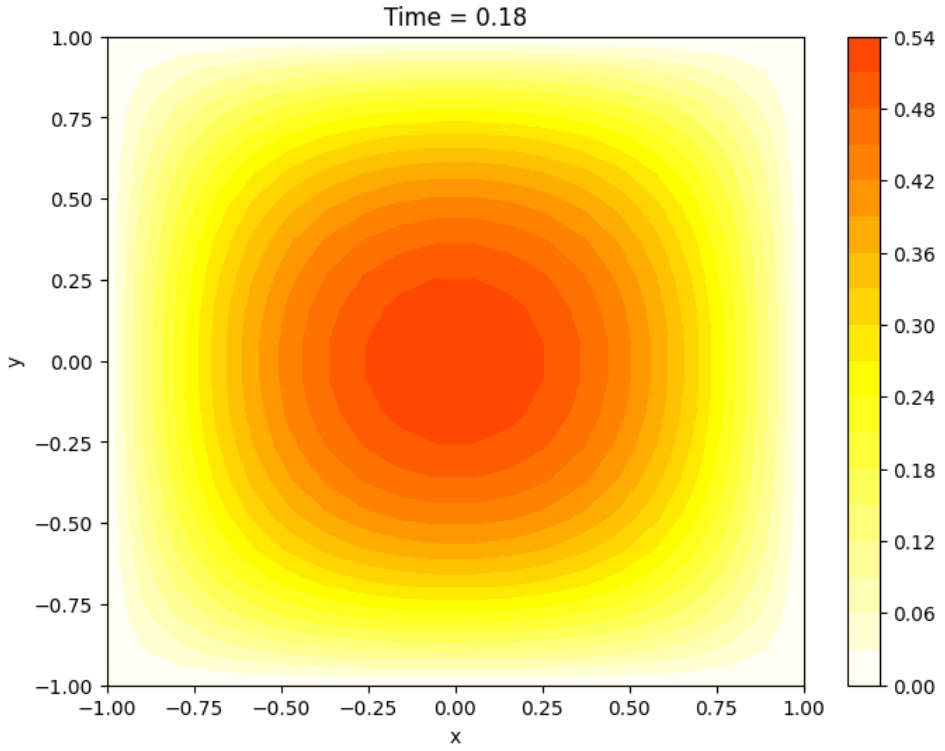


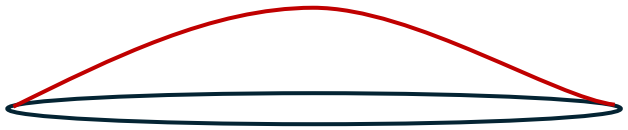
- dt가 감소할수록 기울기 2에 수렴.

Q&A

Problem (2)

N = 20 dt= 0.01





N = 20 dt= 0.001

